

Part 1: Linux Basics & File System Navigation

Linux runs the vast majority of servers, cloud infrastructure, and security tools in the industry. This part builds the foundation everything else depends on: how Linux organizes files, how to move around the terminal, and how to view and manage files and directories.

Step 1: Read the shell prompt

The terminal is your primary interface to Linux. The prompt tells you three things at a glance: who you are, which machine you're on, and where you are in the filesystem. A '\$' means a regular user; a '#' means root — a critical visual cue for privilege level.

```
user@kali:~$ ← username @ hostname : current-dir $
```

Step 2: pwd — print working directory

pwd tells you your exact current location in the filesystem. Get in the habit of checking it before running any command that deletes or moves files.

```
$ pwd /home/alice
```

Step 3: ls — list directory contents

ls shows what's inside a directory. -l gives a long listing (permissions, owner, size, date); -a shows hidden dot-files; -la combines both and is the version analysts use constantly to spot anything unexpected.

```
$ ls -la /etc
$ ls -lh /var/log      # human-readable sizes
$ ls -R /etc/ssh      # recursive listing
```

Do this: Run `ls -la` on your home directory and identify at least two hidden files.

Step 4: cd — change directory

cd moves you around the tree using either an absolute path (starts at /) or a relative path (starts from where you are). ~ is home, .. is the parent directory, and cd - jumps back to your previous location.

```
$ cd /etc/ssh          # absolute path
$ cd ..               # up one level
$ cd ~                # back home
$ cd -                # previous directory
```

Step 5: tree — visualize directory structure

tree draws the folder hierarchy as a diagram, which is much faster than repeated ls calls for understanding how a directory is laid out. Install it first if it's missing.

```
$ sudo apt install tree
$ tree -L 2 /etc      # limit depth to 2 levels
$ tree -a ~           # include hidden files
```

The Filesystem Hierarchy Standard (FHS)

Unlike Windows' drive letters (C:\, D:\), Linux has one unified tree rooted at /. The FHS defines what belongs where. A few directories matter more than others to a security analyst:

Directory	What lives there	Why it matters
/etc	System-wide configuration	passwd, shadow, sudoers, ssh config – the first place attackers and analysts both look
/var/log	System and application logs	The primary evidence trail for incident investigation
/home	Per-user personal directories	Contains user files, shell configs, and SSH keys
/tmp	Temporary files, wiped on reboot	Writable by everyone – a common attacker staging area
/bin, /sbin	Essential command binaries	Core system and admin commands
/proc	Live kernel & process data	Virtual filesystem – /proc/[PID]/ exposes info on every running process

Absolute vs. Relative Paths

An absolute path always starts from / and works no matter where you currently are. A relative path is interpreted from your current directory — shorter to type, but only correct if you know where you're standing. Both matter for writing reliable scripts and commands.

```
Absolute: cd /var/log/auth.log
Relative: cd ../log/auth.log (only correct from the right starting point)
```

File & Directory Operations

mkdir

Creates new directories. -p creates any missing parent directories in one shot — the standard way to build a nested folder structure.

```
$ mkdir reports
$ mkdir -p investigations/case001/logs
$ mkdir -m 700 private # create with specific permissions
```

touch

Creates an empty file if it doesn't exist, or updates its timestamp if it does. Attackers sometimes abuse touch to manipulate file timestamps and cover their tracks — worth remembering during forensic work.

```
$ touch case001/evidence/network_capture.txt user_accounts.txt timeline.txt
```

cp

Copies files or directories; the original stays in place. Use -r for directories, -p to preserve permissions and timestamps (important when copying evidence), -i to confirm before overwriting.

```
$ cp file.txt backup.txt
$ cp -r mydir/ /tmp/mydir_backup/
$ cp -p evidence.log /cases/case001/ # preserve metadata
```

mv

Moves or renames a file or directory. When source and destination are in the same folder, it acts as a rename. Unlike cp, the original is removed.

```
$ mv oldname.txt newname.txt
$ mv suspicious_file.sh /quarantine/
```

rm

Permanently deletes files — Linux has no recycle bin. -r deletes a directory recursively, -i asks for confirmation (a good habit), -f forces deletion with no prompt at all.

```
$ rm -i file.txt
$ rm -r old_directory/
```

Warning: rm -rf is irreversible

rm -rf /path deletes everything under that path instantly, with no confirmation and no recovery. Never run it with a wildcard or variable you haven't double-checked with ls first — a stray space in rm -rf / would try to erase the entire filesystem.

Viewing File Contents

cat

Prints an entire file to the screen at once. Great for short files; for anything log-sized, it floods the terminal — use less or tail instead.

```
$ cat -n /etc/passwd      # -n adds line numbers, useful for citing evidence
$ cat /etc/hosts
```

less / more

Paginated viewers for large files. less is the better of the two — it scrolls both directions and supports searching with / (forward) and lets you jump to the end with G.

```
$ less /var/log/auth.log
Inside less:  / search_term  n = next match  q = quit
```

head / tail

head shows the first lines of a file, tail shows the last. Because logs grow from the bottom, tail is the one analysts reach for constantly — and tail -f 'follows' a file live, streaming new entries as they're written, which is essential during an active investigation.

```
$ head -n 5 /etc/passwd
$ tail -n 50 /var/log/auth.log
$ tail -f /var/log/auth.log      # live monitoring, Ctrl+C to stop
```

Hidden Files

Any file or directory starting with a dot (.) is hidden from a normal ls — not for security, just to keep configuration out of everyday listings. Several dotfiles matter a great deal in an investigation.

Hidden file	Why it matters
.bash_history	Records every command the user typed — a goldmine for forensic review
.bashrc / .bash_profile	Run on shell start/login — a common place attackers plant persistence
.ssh/authorized_keys	Public keys allowed to log in without a password — check for unauthorized entries during incident response

Security-Critical Files to Know

File	Significance
/etc/passwd	Lists all accounts: username, UID, GID, home dir, shell. World-readable — used by attackers to enumerate usernames
/etc/shadow	Hashed passwords, root-readable only. A prime target after privilege escalation
/etc/hosts	Local hostname-to-IP mappings — malware edits this to redirect legitimate domains
/etc/ssh/sshd_config	SSH server config — misconfiguration here (root login, weak ciphers) is a critical exposure
/var/log/auth.log	Every login attempt, sudo use, and SSH session — first stop in a breach investigation
/var/log/wtmp	Binary login/logout/reboot history — read with the last command, not cat
/var/log/faillog	Per-user failed login counter — read with the faillog command
/var/log/dpkg.log	Package install/removal history — reveals when software (including attacker tools) was installed
/var/log/cron	Scheduled task execution history — reveals unauthorized cron activity

faillog

Reports failed login attempts per user, sourced from /var/log/faillog — a quick way to spot accounts under a brute-force attempt.

```
$ faillog -a
```

Getting Help: man Pages

Every core Linux command ships with a manual page covering every flag it supports. When you forget a flag mid-investigation, this is faster than a web search.

```
$ man ls  
$ man tail  
$ man grep      # q to quit
```

Useful Shell Shortcuts

Shortcut	Effect
Tab	Auto-complete commands and paths
Ctrl+C	Cancel the running command
Ctrl+L	Clear the screen
Up Arrow	Recall the previous command
Ctrl+R	Search command history
Ctrl+A / Ctrl+E	Jump to start / end of the line

Part 1 Mini-Lab: Investigation Workspace & Log Review

Scenario: you're a junior analyst asked to set up a case workspace and review recent authentication activity for anything suspicious.

- Create the structure ~/investigations/case001/{logs,evidence,reports} and case002/{logs,evidence,reports} using a single mkdir -p per case.
- Verify it with tree ~/investigations.
- Inside case001/evidence/, touch three files: network_capture.txt, user_accounts.txt, timeline.txt.
- Copy user_accounts.txt into case002/evidence/ with cp, then rename timeline.txt to incident_timeline.txt with mv.
- cat -n /etc/passwd and explain the 7 colon-separated fields for one entry.
- Run tail -n 50 /var/log/auth.log (or syslog if auth.log is empty) and use less with /Failed to count failed logins and note source IPs.
- Write a 3–5 sentence summary: was anything suspicious, what evidence supports it, and what's the recommended next step?